CFGS DESARROLLO DE APLICACIONES MULTIPLATAFORMA

Unidad 4. Clases y objetos

Fundamentos de la programación orientada a objetos (POO), abstracción, encapsulamiento, instanciación en el Heap, constructores, atributos y métodos de instancia, la referencia this, modificadores de acceso, miembros static, la clase Object (toString, equals, hashCode), records inmutables, enums y depuración con IntelliJ IDEA.



MÓDULO PROFESIONAL
Programación (0485) — 1º DAM

RESULTADOS DE APRENDIZAJE
RA1, RA2, RA3, RA4 (Fundamentos de POO, clases, objetos y encapsulamiento)

DURACIÓN ESTIMADA
28 horas lectivas

ENTORNO OFICIAL
IntelliJ IDEA / Java 25 LTS

Hasta este punto, hemos aprendido a estructurar el flujo de ejecución con sentencias condicionales, bucles iterativos, captura de excepciones y funciones modulares (`public static`). Sin embargo, en el paradigma estructurado tradicional los datos (variables y arrays) y las funciones que los manipulan viven completamente separados. En proyectos de software medianos o grandes, esta separación provoca que el código se vuelva frágil, propenso a inconsistencias y difícil de mantener.

La **Programación Orientada a Objetos (POO)** supone una auténtica revolución conceptual en la ingeniería del software: propone modelar los programas informáticos no como secuencias lineales de instrucciones, sino como **sistemas colaborativos de entidades inteligentes denominadas objetos**, que encapsulan tanto sus **datos internos (estado)** como las **operaciones que pueden realizar (comportamiento)**.

En esta unidad nos adentraremos en el corazón de la POO utilizando **Java 25 LTS** (con compatibilidad plena con Java 21 LTS) y **IntelliJ IDEA** como nuestro entorno de desarrollo profesional.

---

1. El Paradigma de la Programación Orientada a Objetos (POO)

1.1 De la Programación Estructurada a los Objetos

Imagina que desarrollamos una aplicación bancaria con el enfoque estructurado de la Unidad 3. Declararíamos variables independientes:

```
// Enfoque estructurado (separación artificial entre datos y lógica)
String[] titulares = {"Laura Gómez", "Carlos Ruíz"};
String[] iban = {"ES211234...", "ES984321..."};
double[] saldos = {1450.50, 320.00};
boolean[] bloqueadas = {false, true};

// Método externo que debe recibir y coordinar todos los arrays sueltos
public static void transferir(int origenIdx, int destinoIdx, double cantidad) {
    if (!bloqueadas[origenIdx] && saldos[origenIdx] >= cantidad) {
        saldos[origenIdx] -= cantidad;
        saldos[destinoIdx] += cantidad;
    }
}
```

Este enfoque presenta debilidades críticas:

1. **Desincronización de datos:** Si se elimina o reordena un índice en `titulares` , todos los arrays paralelos pueden quedar desalineados.
2. **Falta de integridad:** Cualquier parte del código puede acceder directamente a `saldos[origenIdx] = -50000;` corrompiendo las reglas del negocio financiero sin ningún tipo de control o validación.
3. **Escalabilidad nula:** Añadir una nueva propiedad (como el límite de crédito o el tipo de cuenta) exige modificar decenas de métodos y parámetros en toda la aplicación.

La **POO** resuelve este problema agrupando los datos y las funciones que operan sobre ellos en una única unidad lógica coherente: **la clase** `CuentaBancaria` .

1.2 Los Cuatro Pilares Fundamentales de la POO

El paradigma orientado a objetos se sustenta sobre cuatro principios universales de diseño:

```
+-------------------------------------------------------------------------+
|                 LOS CUATRO PILARES DE LA ORIENTACIÓN A OBJETOS           |
+------------------+--------------------+------------------+-------------+
|    ABSTRACCIÓN   |   ENCAPSULAMIENTO  |     HERENCIA     | POLIMORFISMO|
| Aislar lo esencial| Ocultar el estado  | Reutilizar y     | Tratar a    |
| ignorando detalles| interno y exponer  | extender clases  | diferentes  |
| irrelevantes para | una interfaz       | base jerárquicas | objetos bajo|
| el problema.      | pública segura.    | (se ve en UD5).  | una interfaz|
+------------------+--------------------+------------------+-------------+
```

1. **Abstracción:** Capacidad de identificar las características y comportamientos esenciales de una entidad del mundo real que resultan pertinentes para nuestro sistema, descartando los detalles superfluos. Para una clínica veterinaria, un perro tiene `nombre`, `peso`, `raza` y `fechaVacunacion`; el color de sus ojos o su juguete favorito suelen ser irrelevantes.
2. **Encapsulamiento (*Information Hiding*):** Mecanismo que empaqueta los atributos y métodos dentro de una clase y restringe el acceso directo al estado interno desde el exterior. El objeto actúa como una cápsula protectora: solo se puede interactuar con él a través de métodos públicos controlados.
3. **Herencia (introducción):** Mecanismo de extensión que permite a una clase nueva (subclase) heredar atributos y métodos de una clase preexistente (superclase), promoviendo la reutilización del código (se profundizará en la Unidad 5).
4. **Polimorfismo (introducción):** Capacidad que permite enviar un mismo mensaje (invocar un método) a objetos de diferentes clases y que cada uno responda de forma específica a su propia naturaleza (se profundizará en la Unidad 5).

1.3 Diferencia Fundamental: Clase vs. Objeto

La distinción entre clase y objeto es el concepto más trascendental que debes dominar en esta etapa:

Concepto	Definición Analógica	Naturaleza en Java
Clase (*Class*)	Es el **plano arquitectónico**, la **receta** o el **molde**. Define qué atributos y qué métodos tendrán los elementos creados con ella, pero no ocupa memoria para datos reales.	Código fuente (`.java`), compilado en `.class` y cargado en el *Metaspace*.
Objeto (*Object / Instance*)	Es la **casa construida**, el **pastel horneado** a partir de la receta. Es una entidad concreta y viva con valores específicos para sus atributos.	Instancia física creada con el operador `new` alojada en la memoria **Heap**.

La Tríada del Objeto: Estado, Comportamiento e Identidad

Todo objeto en tiempo de ejecución se caracteriza por tres elementos:

1. **Estado:** Conjunto de valores almacenados en sus variables o atributos de instancia en un instante determinado.
2. **Comportamiento:** Conjunto de métodos y operaciones que el objeto puede ejecutar, capaces de consultar o alterar su estado interno.
3. **Identidad:** Propiedad intrínseca que distingue a un objeto de todos los demás en la memoria, incluso si dos objetos contienen exactamente los mismos valores en todos sus atributos (su dirección física de memoria o referencia en el Heap es única).

1.4 Nociones Básicas de Modelado UML: El Diagrama de Clases

El **Lenguaje Unificado de Modelado (UML - *Unified Modeling Language*)** es el estándar internacional de la industria para representar gráficamente la estructura y relaciones de un sistema software antes de codificarlo.

En UML, una clase se representa mediante un rectángulo dividido en tres compartimentos horizontales:

```
+------------------------------------------------------------+
|                      CuentaBancaria                        | <-- 1. Nombre de la clase
+------------------------------------------------------------+
| - titular: String                                          | <-- 2. Atributos
| - iban: String                                             |    (visibilidad, nombre, tip
o)
| - saldo: double                                            |
| - bloqueada: boolean                                       |
+------------------------------------------------------------+
| + ingresar(cantidad: double): void                         | <-- 3. Métodos
| + retirar(cantidad: double): boolean                       |    (visibilidad, nombre, par
ámetros, retorno)
| + getSaldo(): double                                       |
| + estaBloqueada(): boolean                                 |
+------------------------------------------------------------+
```

Símbolos Universales de Visibilidad en UML:

- `-` **(Privado /** `private` **):** Accesible exclusivamente desde dentro del código de la propia clase.
- `+` **(Público /** `public` **):** Accesible libremente desde cualquier clase del proyecto.
- `#` **(Protegido /** `protected` **):** Accesible dentro del mismo paquete y por subclases hereditarias.
- `~` **(Paquete /** ***package-private*** **):** Accesible únicamente por clases situadas dentro del mismo paquete (sin modificador en Java).

---

2. Definición de Clases en Java

2.1 Anatomía Canónica de una Clase

En Java, cada clase pública debe declararse en su propio fichero con el mismo nombre y extensión `.java` (por ejemplo, `CuentaBancaria.java`):

```
package es.ies.dam.banco; // Declaración del paquete de pertenencia

public class CuentaBancaria {

    // 1. ATRIBUTOS DE INSTANCIA (Estado interno)
    private String titular;
    private String iban;
    private double saldo;
    private boolean bloqueada;

    // 2. MÉTODOS CONSTRUCTORES (Inicialización)
    public CuentaBancaria(String titular, String iban) {
        this.titular = titular;
        this.iban = iban;
        this.saldo = 0.0;
        this.bloqueada = false;
    }

    // 3. MÉTODOS DE INSTANCIA (Comportamiento)
    public void ingresar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a ingresar debe ser positiva: "
+ cantidad);
        }
        if (this.bloqueada) {
            throw new IllegalStateException("Operación denegada: la cuenta está bloquead
a.");
        }
        this.saldo += cantidad;
    }

    public boolean retirar(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad a retirar debe ser positiva: " +
cantidad);
        }
        if (this.bloqueada || this.saldo < cantidad) {
            return false; // Retirada rechazada
        }
        this.saldo -= cantidad;
        return true; // Retirada completada
    }

    // 4. MÉTODOS ACCESORES (Getters)
    public double getSaldo() {
        return this.saldo;
    }

    public String getTitular() {
        return this.titular;
    }

    public boolean isBloqueada() {
        return this.bloqueada;
    }
}
```

2.2 Atributos de Instancia y Valores por Defecto en el Heap

Cuando se declara un atributo a nivel de clase (sin el modificador `static`), cada objeto instanciado tendrá su propia copia independiente de esa variable en el Heap.

A diferencia de las variables locales dentro de un método (que deben inicializarse explícitamente antes de leerse o el compilador arrojará un error), **los atributos de instancia se inicializan automáticamente por la JVM** al reservar memoria con `new`:

Tipo de Atributo	Valor Inicial por Defecto
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`boolean`	`false`
`char`	`'\u0000'` (carácter nulo)
Referencias a Objetos (`String`, arrays, envoltorios)	`null`

2.3 Métodos de Instancia y la Palabra Reservada `this`

Un **método de instancia** es una función que opera en el contexto de un objeto concreto. Dentro de cualquier método no estático de una clase, Java proporciona automáticamente una variable implícita especial: **`this`**.

`this` es una referencia que **apunta directamente a la instancia actual del objeto que está ejecutando el método**.

Usos Principales de `this`:

1. **Desambiguación de nombres (Sombreamiento de variables o *Shadowing*):**
   Cuando un parámetro formal o una variable local tiene exactamente el mismo nombre que un atributo de instancia, el identificador local oculta al atributo. Se utiliza `this.` para indicar inequívocamente al compilador que nos referimos al campo del objeto:

   ```
   public void setTitular(String titular) {
       this.titular = titular; // this.titular = Atributo; titular = Parámetro recibido
   }
   ```

2. **Invocación encadenada de constructores (`this(...)`):**
   Permite a un constructor llamar a otro constructor de la misma clase para evitar duplicación de código de inicialización (analizado en profundidad en la Sección 4).

3. **Paso del propio objeto como argumento:**
   Si un objeto necesita pasarse a sí mismo como argumento a otro método o registrarse en un servicio:

   ```
   gestorNotificaciones.registrarCuenta(this);
   ```

4. **Retorno de la propia instancia para interfaces fluidas (*Method Chaining*):**
   Al hacer que un método devuelva `this`, es posible encadenar múltiples llamadas seguidas en una única sentencia:

```
public CuentaBancaria asignarLimite(double limite) {
    this.limiteCredito = limite;
    return this; // Permite: cuenta.asignarLimite(500).ingresar(100);
}
```

2.4 Las Variables Objeto son Referencias (Continente vs. Contenido)

Cuando declaras una variable de tipo primitivo (`int edad = 8;`), la variable almacena directamente los bits del valor numérico en la pila (*Stack*). Sin embargo, en el caso de los objetos, **declarar una variable no crea el objeto ni lo almacena dentro de ella**: lo que guarda la variable es una **referencia** (un puntero o dirección de memoria) que señala el lugar del **Heap** donde vive realmente el objeto.

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    public void presentar() {
        System.out.println("Hola, me llamo " + this.nombre + " y tengo " + this.edad + " año
s.");
    }

    public String getNombre() { return this.nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
}
```

¿Qué ocurre cuando asignamos una variable objeto a otra (`p2 = p1`)?

Observa con mucha atención el siguiente fragmento de código:

```
Persona p1 = new Persona("Ada", 36);
Persona p2 = p1; // ¡NO se duplica el objeto! Se copia únicamente la referencia (dirección)

p2.setNombre("Grace");
System.out.println(p1.getNombre()); // Imprime: "Grace"
```

Asignar Referencias NO Clona Objetos

Cuando asignas una variable objeto a otra (`Persona p2 = p1;`), **no se crea una copia del objeto en el Heap**, sino que se copia la dirección de memoria. Ambas variables (`p1` y `p2`) acaban apuntando exactamente **al mismo objeto físico**. Por tanto, cualquier modificación realizada a través de `p2` será inmediatamente visible al consultar `p1` .

```
       MEMORIA STACK                          MEMORIA HEAP
+------------------------+           +----------------------------------+
| Variable: p1 (0x4a10)  | --------> | Objeto Persona (0x4a10)          |
+------------------------+     ▲     | - nombre: "Grace" (antes "Ada")  |
                               |     | - edad: 36                       |
+------------------------+     |     +----------------------------------+
| Variable: p2 (0x4a10)  | ----+
+------------------------+
```

La Metáfora del Continente y el Contenido (`==` frente al estado interno)

Imagina dos recipientes físicos distintos: una **copa** y una **taza**, ambos llenos con el mismo líquido (`0xCAFE`):

```
public class Recipiente {
    private int contenido = 0xCAFE;

    public static void main(String[] args) {
        Recipiente copa = new Recipiente();
        Recipiente taza = new Recipiente();

        System.out.println("¿Mismo continente (copa == taza)? " + (copa == taza)); // false
        System.out.println("Contenido de la copa: " + copa.contenido);             // 51966
(0xCAFE)
        System.out.println("Contenido de la taza: " + taza.contenido);             // 51966
(0xCAFE)
    }
}
```

- El operador `==` aplicado a referencias pregunta: **«¿Son exactamente el mismo recipiente físico?»** (`false`, porque se crearon con dos `new` distintos).
- Para saber si dos recipientes distintos albergan **el mismo contenido**, nunca debemos usar `==`, sino el método `.equals()` (que estudiaremos en profundidad en la Sección 6).

La Referencia Nula (`null`) y `NullPointerException`

Cuando una variable de tipo objeto no apunta a ninguna instancia en la memoria Heap, decimos que contiene una **referencia nula (`null`)**:

```
Persona p3 = null; // p3 no apunta a ningún objeto vivo

if (p3 != null) {
    p3.presentar(); // Comprobación defensiva segura
}
```

Si intentas acceder a un atributo o invocar un método sobre una variable cuyo valor es `null` (por ejemplo, `p3.presentar()`), la JVM abortará la ejecución de inmediato lanzando la célebre excepción **`java.lang.NullPointerException`**.

---

3. Control de Visibilidad, Encapsulamiento y Ampliación de Métodos

El **encapsulamiento** es el escudo protector que garantiza la integridad del software. Consiste en declarar los atributos como **`private`** para que ningún código exterior pueda manipularlos directamente de forma arbitraria, y

proporcionar métodos públicos (`getters` y `setters`) con lógica de validación.

3.1 La Tabla Maestra de Modificadores de Acceso en Java

Java ofrece cuatro niveles de visibilidad para clases, atributos y métodos:

Modificador	Misma Clase	Mismo Paquete	Subclases (otro paquete)	Todo el Proyecto
`private`	**SÍ**	No	No	No
(default / package-private)	**SÍ**	**SÍ**	No	No
`protected`	**SÍ**	**SÍ**	**SÍ** (vía herencia)	No
`public`	**SÍ**	**SÍ**	**SÍ**	**SÍ**

> **El Modificador por Defecto (Package-Private)**
> Si no antepones ninguna palabra clave de visibilidad antes de un atributo o método (por ejemplo `double saldo;`), Java no lo hace privado ni público: aplica el nivel **por defecto o package-private**, permitiendo que cualquier otra clase situada dentro de la misma carpeta de paquete pueda leerlo y modificarlo directamente. En POO profesional, **declara siempre explícitamente `private` tus atributos**.

3.2 Métodos Accesores (*Getters*) y Mutadores (*Setters*)

Para permitir la lectura o actualización segura de atributos privados, la convención estándar de Java (*JavaBeans standard*) define:

- **Getters (Lectura):** Prefijo `get` seguido del nombre del atributo con la primera letra en mayúscula (ej. `getTitular()`). En variables booleanas es convención estándar usar el prefijo `is` (ej. `isActivo()` , `isAgotado()` , `isBloqueada()`).
- **Setters (Escritura controlada):** Prefijo `set` seguido del nombre del atributo. Deben incorporar validaciones de invariantes para proteger al objeto frente a estados corruptos.
- **Getters y Setters para Arrays:** Cuando una clase encapsula un array (como `private String[] autores`), además del getter/setter de la colección completa es muy buena práctica ofrecer métodos indexados para consultar o modificar un elemento individual (`getAutor(int i)` y `setAutor(int i, String autor)`).

```
public class Alumno {
    private String nombre;
    private double nota; // Invariante: debe estar entre 0.0 y 10.0

    public String getNombre() {
        return this.nombre;
    }

    public void setNombre(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            throw new IllegalArgumentException("El nombre del alumno no puede ser nulo ni va
cío.");
        }
        this.nombre = nombre.trim();
    }

    public double getNota() {
        return this.nota;
    }

    public void setNota(double nota) {
        if (nota < 0.0 || nota > 10.0) {
            throw new IllegalArgumentException("Nota no válida (" + nota + "). Debe estar en
el rango [0.0, 10.0].");
        }
        this.nota = nota;
    }
}
```

Generación Automática en IntelliJ IDEA

Aunque ahora estamos aprendiendo y hay que crearlos a mano, muchos IDEs nos permiten automatizar esta tarea. En IntelliJ IDEA, coloca el cursor dentro de la clase y presiona `Alt + Insert` (o haz clic derecho y selecciona *Generate...*). Elige **Getter and Setter**, selecciona los atributos que deseas encapsular y el IDE generará el código de forma instantánea.

3.3 Atributos Inmutables con la Palabra Reservada `final`

Cuando un atributo de instancia se declara con la palabra clave `final`, solo se le puede asignar valor una única vez (ya sea en su propia declaración o en el constructor de la clase). Tras esa asignación inicial, su valor permanece inalterable durante toda la vida útil del objeto:

```
import java.util.Objects;

public class Empleado {
    private final String dni; // Inmutable: el DNI nunca cambia tras instanciarse
    private String departamento;
    private double sueldoBase;

    public Empleado(String dni, String departamento, double sueldoBase) {
        this.dni = Objects.requireNonNull(dni, "El DNI no puede ser nulo");
        this.departamento = departamento;
        this.sueldoBase = sueldoBase;
    }

    public String getDni() {
        return this.dni; // Proporciona getter pero NO setter
    }

    public String getDepartamento() {
        return this.departamento;
    }

    public void setDepartamento(String departamento) {
        this.departamento = departamento;
    }
}
```

3.4 La Interfaz Pública de una Clase y la Reducción del Acoplamiento

El conjunto de todos los métodos `public` de una clase constituye su **interfaz pública**: el contrato visible a través del cual el resto del programa puede comunicarse con los objetos de esa clase sin necesidad de saber cómo están programados por dentro.

```
   ❌ SIN ENCAPSULAMIENTO (Alto Acoplamiento)
   [Código Externo] -------- acceso directo --------> [Atributos internos: saldo, eval1...]
   (Si cambia el tipo o nombre del atributo interno, se rompe todo el código externo)

   ✅ CON INTERFAZ PÚBLICA (Bajo Acoplamiento)
   [Código Externo] ---> [Interfaz Pública: getMedia(), setNota()] ---> [Estado Interno Priv
ado]
   (La implementación interna puede cambiar libremente sin romper ningún programa externo)
```

Ocultar los detalles internos tras una interfaz pública de getters y setters aporta tres ventajas de ingeniería decisivas:

1. **Propiedades calculadas al vuelo:** Un getter no tiene por qué corresponderse con una variable física guardada en memoria. Por ejemplo, `getMedia()` puede calcular en tiempo real `(eval1 + eval2 + eval3) / 3.0`, o `getPrecioConIva()` devolver `this.precio * 1.21`.
2. **Validación y seguridad centralizada:** Cualquier intento de lectura o escritura pasa por un único punto de control donde podemos verificar permisos o lanzar `IllegalArgumentException` si el dato es inválido.
3. **Desacoplamiento y mantenibilidad:** Si dos años después decides cambiar la estructura interna de la clase (por ejemplo, sustituir tres variables `eval1, eval2, eval3` por un array `double[] evaluaciones`), ningún programa externo se romperá mientras mantengas intacta la firma de los métodos públicos `getEval1()`, `getMedia()`, etc.

3.5 Diseño de Métodos en Clases: Responsabilidad, Métodos Auxiliares Privados y Sobrecarga en POO

En la Unidad 3 aprendimos a estructurar métodos y funciones modulares (`public static`), el paso de parámetros por valor y los conceptos básicos de sobrecarga y recursividad. En la Programación Orientada a Objetos, los métodos dejan de ser funciones procedimentales aisladas para convertirse en las **capacidades operativas que dan vida al objeto** y gobiernan su estado interno.

1. El Principio de Responsabilidad Única y Métodos de Instancia

Un método de instancia debe cumplir un único propósito claro dentro del dominio de la clase:

- **Consultar estado:** Métodos que interrogan los atributos del objeto y devuelven una respuesta calculada o directa (como `getSaldo()` , `esMoroso()` o `calcularArea()`).
- **Mutar estado con validación:** Métodos que alteran los atributos asegurando que nunca se vulneren las reglas de integridad del objeto (como `ingresar(cantidad)` o `frenar(decremento)`).
- **Notificar o coordinar:** Métodos que permiten a la entidad interactuar con otros objetos colaboradores.

2. Métodos Auxiliares Privados (*Private Helper Methods*)

Una de las mejores prácticas de la ingeniería del software es mantener la **interfaz pública lo más reducida y limpia posible** (*Principio de Mínima Exposición*). Si una operación pública requiere cálculos matemáticos complejos, algoritmos de verificación o pasos intermedios, estos no deben exponerse al exterior: deben encapsularse en **métodos privados auxiliares**:

```
public class CuentaBancaria {
    private String titular;
    private String iban;
    private double saldo;

    // MÉTODO PÚBLICO: Interfaz visible y clara para el cliente de la clase
    public void ingresar(double cantidad) {
        validarCantidadPositiva(cantidad); // Delega la validación en un método privado
        this.saldo += cantidad;
    }

    public void retirar(double cantidad) {
        validarCantidadPositiva(cantidad);
        validarSaldoDisponible(cantidad);
        this.saldo -= cantidad;
    }

    // MÉTODOS PRIVADOS AUXILIARES: Ocultos tras la cápsula de la clase
    private void validarCantidadPositiva(double cantidad) {
        if (cantidad <= 0) {
            throw new IllegalArgumentException("La cantidad debe ser estrictamente positiva:
" + cantidad);
        }
    }

    private void validarSaldoDisponible(double cantidad) {
        if (cantidad > this.saldo) {
            throw new IllegalArgumentException("Saldo insuficiente. Disponible: " + this.sal
do + ", Solicitado: " + cantidad);
        }
    }
}
```

Ventajas del método auxiliar privado:

- Evita la duplicación de código (*DRY - Don't Repeat Yourself*): varias operaciones públicas pueden reutilizar la misma comprobación interna.
- Si la regla de validación cambia en el futuro, solo se modifica ese método privado sin alterar ningún programa externo que use la clase.

3. Sobrecarga de Métodos de Objeto y Delegación

Como vimos en la Unidad 3, la **sobrecarga (*Overloading*)** consiste en definir dos o más métodos con el mismo nombre pero distinta firma (distinto número o tipos de parámetros) dentro de la misma clase, resolviéndose mediante **ligadura estática** en tiempo de compilación.

En el diseño de clases orientadas a objetos, el patrón canónico de sobrecarga consiste en hacer que los métodos más sencillos **deleguen en el método más completo**, proporcionando valores predeterminados:

```
public class Notificador {
    // Método sobrecargado básico: proporciona un asunto por defecto
    public void enviarMensaje(String destinatario, String cuerpo) {
        enviarMensaje(destinatario, "Notificación del Sistema", cuerpo); // Delega en el mét
odo completo
    }

    // Método completo: concentra toda la lógica real de envío
    public void enviarMensaje(String destinatario, String asunto, String cuerpo) {
        if (destinatario == null || destinatario.isBlank()) {
            throw new IllegalArgumentException("El destinatario no puede estar vacío");
        }
        System.out.printf("Enviando a [%s] con asunto '%s': %s%n", destinatario, asunto, cue
rpo);
    }
}
```

4. Colaboración entre Objetos: Métodos que Operan con Otras Instancias

Los objetos rara vez trabajan solos; su verdadero poder radica en la colaboración. Un método de instancia puede recibir como argumento otra instancia de su misma clase o de otra clase:

```
public class CuentaBancaria {
    private String titular;
    private double saldo;

    // Colaboración entre dos cuentas bancarias
    public void transferir(CuentaBancaria cuentaDestino, double cantidad) {
        if (cuentaDestino == null) {
            throw new IllegalArgumentException("La cuenta de destino no puede ser nula");
        }
        if (cuentaDestino == this) {
            throw new IllegalArgumentException("No puedes realizar una transferencia a la mi
sma cuenta");
        }

        // Reutiliza los métodos validados de la propia clase
        this.retirar(cantidad);
        cuentaDestino.ingresar(cantidad);
    }
}
```

> **Paso de Referencias por Valor**
>
> Recuerda el principio fundamental consolidado en la Unidad 3: en Java **todo se pasa por valor**. Al pasar `cuentaDestino` al método, se recibe una **copia de la dirección de memoria**. A través de esa referencia compartida podemos alterar el estado del objeto en el Heap (`cuentaDestino.ingresar(cantidad)`), pero si dentro del método reasignamos `cuentaDestino = null`, solo desconectamos la variable local del método, dejando intacta la variable original del exterior.

5. Recursividad Encapsulada como Comportamiento de Objeto

En la Unidad 3 estudiamos los fundamentos teóricos de la recursividad (el caso base y la pila de llamadas o *Call Stack*). En Programación Orientada a Objetos, los algoritmos recursivos a menudo se encapsulan como

comportamientos internos de una clase, permitiendo que el objeto mantenga protegidas sus métricas internas de ejecución (como contadores de pasos o historiales):

```
public class SolucionadorHanoi {
    private int totalMovimientos; // Estado interno protegido del objeto

    public SolucionadorHanoi() {
        this.totalMovimientos = 0;
    }

    public int getTotalMovimientos() {
        return totalMovimientos;
    }

    // Método público de entrada para iniciar la resolución
    public void resolver(int numDiscos, char origen, char auxiliar, char destino) {
        this.totalMovimientos = 0; // Reinicia el estado para cada ejecución
        ejecutarHanoi(numDiscos, origen, auxiliar, destino);
    }

    // Método recursivo privado que realiza el trabajo algorítmico interno
    private void ejecutarHanoi(int n, char desde, char aux, char hasta) {
        if (n == 1) {
            this.totalMovimientos++;
            System.out.printf("Paso %d: Mover disco 1 de %c a %c%n", totalMovimientos, desd
e, hasta);
            return; // Caso base
        }
        ejecutarHanoi(n - 1, desde, hasta, aux);
        this.totalMovimientos++;
        System.out.printf("Paso %d: Mover disco %d de %c a %c%n", totalMovimientos, n, desd
e, hasta);
        ejecutarHanoi(n - 1, aux, desde, hasta);
    }
}
```

---

4. Ciclo de Vida de los Objetos: Instanciación, Constructores y Destrucción

Un objeto en Java no nace por generación espontánea ni permanece para siempre en la memoria. Atraviesa un ciclo de vida bien definido: **nacimiento (instanciación con constructores)**, **vida activa (invocación de métodos y modificación de estado)** y **muerte (pérdida de referencias y reciclaje por el Garbage Collector)**.

4.1 El Operador `new` y la Reserva de Memoria en el Heap

Cuando el compilador y la JVM se encuentran con una sentencia como:

```
CuentaBancaria miCuenta = new CuentaBancaria("Sofía Morales", "ES9121000418450200051332");
```

La JVM ejecuta internamente una secuencia de cuatro pasos atómicos:

1. **Reserva física de espacio:** Se solicita un bloque contiguo de bytes en la memoria **Heap** con capacidad suficiente para albergar todos los atributos de instancia declarados en la clase (y los heredados de sus superclases).

2. **Inicialización por defecto:** Todas las variables del objeto recién reservado se inicializan a sus valores neutros o por defecto (`0` , `0.0` , `false` , `null`).
3. **Ejecución del constructor:** Se invoca el método constructor correspondiente, pasando los argumentos facilitados. En este paso se ejecutan las asignaciones iniciales y las validaciones de negocio.
4. **Devolución de la referencia:** El operador `new` devuelve la dirección física de memoria donde reside el objeto en el Heap (por ejemplo, `0x7b4a2f`), la cual queda almacenada en la variable local de la pila **Stack** (`miCuenta`).

```
      MEMORIA STACK                          MEMORIA HEAP
+------------------------+            +-----------------------------------+
| Variable: miCuenta     | ---------> | Objeto CuentaBancaria (0x7b4a2f)  |
| (Dirección 0x7b4a2f)   |            | - titular: "Sofía Morales"        |
+------------------------+            | - iban: "ES91210004..."           |
                                      | - saldo: 0.0                      |
                                      | - bloqueada: false                |
                                      +-----------------------------------+
```

4.2 Anatomía y Propósito de los Constructores

Un **constructor** es un método especial cuya única misión es inicializar un objeto en un estado coherente, válido y listo para operar desde el mismo instante en que se reserva su memoria.

Reglas Sintácticas Inquebrantables de los Constructores:

1. Su nombre debe coincidir **exactamente con el nombre de la clase** (respetando estrictamente mayúsculas y minúsculas).
2. **No tiene ningún tipo de retorno**, ni siquiera `void` . *(Si colocas `public void CuentaBancaria()` , Java lo interpretará como un método ordinario con un nombre desafortunado, no como un constructor).*
3. Se invoca única y exclusivamente mediante la palabra reservada `new` en el momento de crear el objeto.

4.3 El Constructor por Defecto Implícito (*Default Constructor*)

Si escribes una clase y **no declaras ningún constructor explícito**, el compilador de Java inserta automáticamente en el bytecode un constructor público sin parámetros y con cuerpo vacío:

```
public class Sensor {
    private double temperatura;
    // El compilador añade automáticamente: public Sensor() {}
}
```

> **La Desaparición del Constructor por Defecto**
>
> En el momento exacto en que declaras **cualquier constructor propio con parámetros**, el compilador de Java **retira de inmediato el constructor por defecto sin parámetros**.
>
> Si tras definir `public Sensor(double tempInicial)` intentas hacer `Sensor s = new Sensor();` , el código fallará en compilación con el error: `constructor Sensor in class Sensor cannot be applied to given types` . Si necesitas ambos, debes declarar explícitamente el constructor sin argumentos.

4.4 Sobrecarga de Constructores y Delegación con `this(...)`

Al igual que los métodos ordinarios, los constructores admiten **sobrecarga**: podemos proporcionar múltiples constructores con diferente número o tipo de parámetros para permitir distintas formas de inicializar un objeto.

Para evitar la duplicación de código de inicialización y validación entre constructores, Java permite que un constructor invoque a otro de su misma clase utilizando la sintaxis `this(argumentos)`:

```
public class Producto {
    private final String codigo;
    private String descripcion;
    private double precio;
    private int stock;

    // 1. Constructor Principal (Canónico): recibe todos los datos y valida
    public Producto(String codigo, String descripcion, double precio, int stock) {
        if (codigo == null || codigo.trim().isEmpty()) {
            throw new IllegalArgumentException("El código de producto es obligatorio.");
        }
        if (precio < 0 || stock < 0) {
            throw new IllegalArgumentException("El precio y el stock no pueden ser negativo
s.");
        }
        this.codigo = codigo.trim().toUpperCase();
        this.descripcion = (descripcion != null) ? descripcion.trim() : "Sin descripción";
        this.precio = precio;
        this.stock = stock;
    }

    // 2. Constructor Secundario: precio dado, stock inicial por defecto a 0
    public Producto(String codigo, String descripcion, double precio) {
        this(codigo, descripcion, precio, 0); // Delega en el constructor principal
    }

    // 3. Constructor Mínimo: producto recién catalogado con precio 0.0 y stock 0
    public Producto(String codigo, String descripcion) {
        this(codigo, descripcion, 0.0, 0); // Delega en el constructor principal
    }
}
```

> **Regla de Oro de `this(...)`**
>
> La llamada delegada a otro constructor mediante `this(...)` debe ser **estrictamente la primera sentencia ejecutable** del cuerpo del constructor. Colocar cualquier otra instrucción antes de `this(...)` provocará un error de compilación inmediato (`call to this must be first statement in constructor`).

4.5 Bloques de Inicialización y Orden Cronológico Exacto

Además de los constructores, Java permite definir **bloques de inicialización de instancia** (`{ ... }`) y **bloques de inicialización estáticos** (`static { ... }`) directamente en el cuerpo de la clase.

Cuando se utiliza una clase por primera vez y se instancia un objeto con `new`, la JVM respeta un orden cronológico estricto:

1. **Carga de la clase (una sola vez):** Se inicializan las variables `static` (con su valor por defecto o en línea) y se ejecutan los bloques `static { ... }` en orden de aparición.
2. **Reserva en el Heap:** Al ejecutar `new`, se reserva memoria para el objeto y todos sus atributos de instancia toman primero su valor por defecto (`0`, `false`, `null`).
3. **Inicialización en línea y bloques `{ ... }` de instancia:** Se asignan los valores declarados en la propia línea del atributo y se ejecutan los bloques `{ ... }` de arriba abajo.
4. **Cuerpo del constructor:** Finalmente se ejecuta el código dentro del constructor invocado.

```
public class AlumnoCFGS {
    private static int edadMinima = 18;
    private int nia;
    private String nombre;
    private int[] notas = new int[3];
    private boolean repetidor;

    // 1. Bloque estático: se ejecuta una única vez al cargar la clase en la JVM
    static {
        edadMinima--; // Pasa de 18 a 17
    }

    // 2. Bloque de instancia: se ejecuta cada vez que nace un objeto, ANTES del constructor
    {
        this.nia = (int) (Math.random() * 900000) + 100000;
    }

    // 3. Constructor: último paso de inicialización
    public AlumnoCFGS(String nombre) {
        this.nombre = nombre;
    }
}
```

Tras ejecutar `AlumnoCFGS a1 = new AlumnoCFGS("Pepe");`, el estado resultante es: `edadMinima = 17`, `nia` aleatorio de 6 cifras, `nombre = "Pepe"`, `notas = {0, 0, 0}` y `repetidor = false`.

4.6 Destrucción de Objetos y el Recolector de Basura (*Garbage Collector*)

En lenguajes de bajo nivel como C o C++, el programador debe liberar manualmente la memoria con `free()` o `delete`. Un descuido produce fugas de memoria (*memory leaks*) que saturan el sistema.

Java utiliza un modelo de gestión automática de memoria mediante el **Garbage Collector (GC)**, un hilo de fondo de la JVM que monitoriza constantemente el Heap.

¿Cuándo es un Objeto Elegible para Destrucción?

Un objeto se convierte en candidato a ser eliminado cuando **deja de ser alcanzable por cualquier hilo de ejecución activo** (es decir, ya no existe ninguna variable de referencia en el Stack que apunte a él, directa o indirectamente):

```
public static void demostracionMemoria() {
    CuentaBancaria c1 = new CuentaBancaria("Juan", "ES111");
    CuentaBancaria c2 = new CuentaBancaria("Ana", "ES222");

    c1 = c2;
    // Ahora c1 apunta al objeto de Ana.
    // El objeto original de Juan ("ES111") ha quedado HUÉRFANO en el Heap:
    // Ya ninguna variable apunta a él. Es candidato inmediato para el Garbage Collector.

    c2 = null;
    // El objeto de Ana sigue vivo porque c1 aún lo referencia.
}
// Al salir del método demostracionMemoria(), c1 desaparece del Stack.
// Ahora el objeto de Ana también queda huérfano y será reclamado por el GC.
```

El Método finalize() está Prohibido en Java Moderno

Históricamente, la clase `Object` incluía un método `finalize()` que se ejecutaba supuestamente antes de que el GC destruyera el objeto. Sin embargo, su comportamiento impredecible, bloqueos de hilos y graves problemas de seguridad hicieron que Oracle lo declarase **obsoleto desde Java 9 y marcado para eliminación definitiva**.

Para liberar recursos del sistema (archivos abiertos, conexiones a bases de datos, sockets de red), la práctica oficial y obligatoria es implementar la interfaz `java.lang.AutoCloseable` y utilizar la estructura `try-with-resources` vista en la Unidad 3.

---

5. Miembros Estáticos: La Palabra Reservada `static` y Combinaciones con `final`

Hasta ahora, todos los atributos y métodos estudiados pertenecían a **instancias individuales**: cada objeto tenía su propio `saldo` y su propio `titular`.

Sin embargo, en multitud de escenarios necesitamos elementos que no pertenezcan a ningún objeto concreto, sino a **la clase en su conjunto**. Para este propósito, Java proporciona el modificador `static`.

5.1 Variables Estáticas (Atributos de Clase)

Una **variable estática** es un campo declarado con `static`. Existe **una única copia física** de esa variable en toda la JVM (alojada en la zona de memoria *Metaspace*), compartida solidariamente por todos los objetos de esa clase:

```
public class Empleado {
    // 1. Atributo estático: compartido por todos los empleados
    private static int contadorTotalEmpleados = 0;

    // 2. Atributos de instancia: propios de cada empleado individual
    private final int idEmpleado;
    private String nombre;

    public Empleado(String nombre) {
        this.nombre = nombre;
        // Incrementamos el contador global y asignamos un ID secuencial automático
        contadorTotalEmpleados++;
        this.idEmpleado = contadorTotalEmpleados;
    }

    public static int getContadorTotalEmpleados() {
        return contadorTotalEmpleados;
    }

    public int getIdEmpleado() {
        return this.idEmpleado;
    }
}
```

Si instanciamos tres objetos:

```
Empleado e1 = new Empleado("Marina"); // id: 1
Empleado e2 = new Empleado("Roberto"); // id: 2
Empleado e3 = new Empleado("Elena");   // id: 3

System.out.println(Empleado.getContadorTotalEmpleados()); // Imprime: 3
```

5.2 Métodos Estáticos y Reglas de Acceso entre Contextos

Un **método estático** es una función asociada a la clase, no a ningún objeto. Se invoca directamente utilizando el nombre de la clase (`Clase.metodo()`), sin necesidad de crear ninguna instancia con `new` :

```
double hipotenusa = Math.sqrt(Math.pow(3, 2) + Math.pow(4, 2)); // Métodos estáticos de jav
a.lang.Math
```

La Regla de Oro entre Contexto Estático y Contexto de Instancia:

Puesto que un método estático se ejecuta a nivel de clase (incluso cuando se han creado 0 objetos en el Heap):

1. **Dentro de un método `static` NO existe la referencia `this`** .
2. **Un método `static` NO puede acceder directamente a atributos ni métodos de instancia** (salvo que instancie primero un objeto con `new` o lo reciba por parámetro).
3. **Un método de instancia SÍ puede acceder libremente tanto a miembros de instancia (`this`) como a miembros `static` de su clase**.

```
public class EjemploContextos {
    private static int variableDeClase = 1;
    private int variableDeObjeto = 2;

    public static void metodoDeClase() {
        variableDeClase = 3;      // ✅ OK: acceso de estático a estático
        // variableDeObjeto = 2;  // ❌ ERROR DE COMPILACIÓN: ¿de qué objeto si no hay 'thi
s'?
        // metodoDeObjeto();      // ❌ ERROR DE COMPILACIÓN

        // Solo podemos acceder a miembros de objeto si referenciamos una instancia concret
a:
        EjemploContextos obj = new EjemploContextos();
        obj.variableDeObjeto = 20; // ✅ OK: indicamos explícitamente el objeto 'obj'
    }

    public void metodoDeObjeto() {
        variableDeClase = 10;     // ✅ OK: un objeto conoce a su clase
        metodoDeClase();          // ✅ OK
        this.variableDeObjeto = 5;// ✅ OK: acceso al propio objeto actual (this)
    }
}
```

5.3 Tabla Maestra: Combinaciones de `static` y `final` (Matriz 2×2)

Los modificadores `static` y `final` responden a dos preguntas completamente distintas pero complementarias:

- `static` responde a **«¿a quién pertenece?»**: sin `static` pertenece a cada objeto individual; con `static` pertenece a la clase y se comparte entre todas las instancias.
- `final` responde a **«¿puede cambiar de valor?»**: sin `final` puede reasignarse; con `final` solo admite una única asignación inicial.

Al cruzar ambos modificadores obtenemos cuatro combinaciones fundamentales en el diseño en Java:

	Sin `static` *(uno por cada objeto)*	**Con `static` *(uno compartido en la clase)***
Sin `final` *(puede cambiar)*	**1. Atributo de instancia normal** `private String nombre;` Cada objeto tiene el suyo y puede cambiar a lo largo de su vida.	**3. Variable de clase compartida** `private static int contador = 0;` Común a todos los objetos y su valor varía durante la ejecución.
Con `final` *(no puede cambiar)*	**2. Constante de instancia** `private final String dni;` Se asigna en el constructor de cada objeto y ya no cambia jamás para ese objeto.	**4. Constante global de clase** `public static final double IVA = 0.21;` Valor único, fijo y universal para toda la clase. Se escribe en `MAYUSCULAS`.

```
public class Factura {
    // 4. Constante de clase (static + final): única e inmutable para todo el programa
    public static final double IVA_GENERAL = 0.21;

    // 3. Variable de clase (static sin final): compartida pero mutable
    private static int totalFacturasEmitidas = 0;

    // 2. Constante de instancia (final sin static): distinta en cada factura, pero fija tra
s nacer
    private final int numeroFactura;

    // 1. Atributo de instancia normal (sin static ni final): propio de cada objeto y modifi
cable
    private String concepto;

    public Factura(String concepto) {
        totalFacturasEmitidas++;
        this.numeroFactura = totalFacturasEmitidas; // Se asigna una sola vez y queda sellad
o
        this.concepto = concepto;
    }
}
```

> **Cuándo NO usar `static`**
>
> No utilices variables `static` como «atajo» para compartir datos entre métodos sin pasarlos por parámetro. Una variable `static` mutable sin justificación real es, en la práctica, una **variable global encubierta** que rompe el encapsulamiento, acopla las clases entre sí y provoca errores gravísimos difíciles de rastrear. Usa `static` únicamente para **constantes (`public static final`)**, **contadores/registros de clase** o **métodos de utilidad pura** (como `Math.sqrt()`).

---

6. La Clase Raíz del Universo Java: `java.lang.Object`

En la jerarquía de tipos de Java, **toda clase desciende directa o indirectamente de la clase universal `java.lang.Object`**. Si no especificas la cláusula `extends` al declarar una clase, el compilador añade automáticamente `extends Object`.

Esto garantiza que cualquier objeto en Java herede un conjunto de métodos fundamentales:

Método en `java.lang.Object`	Propósito	¿Se puede / debe sobrescribir?
`String toString()`	Devuelve la representación textual del objeto. Por defecto devuelve `Clase@hashHex`.	**Sí (Recomendado siempre)**
`boolean equals(Object obj)`	Comprueba si dos objetos contienen la misma información semántica. Por defecto evalúa `this == obj`.	**Sí (Según identidad de negocio)**
`int hashCode()`	Devuelve el código hash entero del objeto para colecciones (`HashSet`, `HashMap`).	**Sí (Obligatorio si se sobrescribe `equals`)**
`protected Object clone()`	Crea y devuelve una copia superficial (*shallow copy*) del objeto (requiere `Cloneable`).	Se puede (se prefiere constructor de copia)
`final Class<?> getClass()`	Devuelve el objeto `Class` que representa la clase real en tiempo de ejecución.	**No (`final`)**
`final void notify() / notifyAll() / wait()`	Sincronización y coordinación de hilos concurrentes (*threads*).	**No (`final`)**
`protected void finalize()`	Invocado históricamente por el GC antes de destruir el objeto.	**Obsoleto / Prohibido desde Java 9+**

6.1 Sobrescritura (*Override*) y la Anotación `@Override`

La **sobrescritura** consiste en redefinir en una subclase un método heredado de su superclase para dotarlo de un comportamiento específico.

Para indicar formalmente al compilador que nuestra intención es sobrescribir un método, anteponemos la anotación `@Override`:

```
@Override
public String toString() { ... }
```

> **¿Por qué es indispensable usar @Override?**
> Si cometes una errata al escribir el nombre del método (por ejemplo `public String tostring()` o `public boolean equals(Cliente c)` en vez de `equals(Object obj)`), sin `@Override` el compilador creerá que estás sobrecargando o creando un método nuevo y tu clase no sustituirá el comportamiento heredado de `Object`. Con `@Override`, el compilador verifica la firma exacta y detiene la compilación informándote del error al instante.

6.2 Sobrescribir `toString()`: Representación Textual Legible

La implementación por defecto de `toString()` en la clase `Object` es:

```
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

Por ello, imprimir un objeto sin `toString()` produce una salida críptica como `es.ies.dam.banco.CuentaBancaria@7b4a2f`. Al sobrescribirlo:

```
@Override
public String toString() {
    return String.format("CuentaBancaria[IBAN='%s', Titular='%s', Saldo=%.2f€, Bloqueada=%
b]",
            this.iban, this.titular, this.saldo, this.bloqueada);
}
```

Ahora, al pasar el objeto a `System.out.println(miCuenta)` o concatenarlo con un `String` (`"Cuenta: " + miCuenta`), Java invoca automáticamente nuestro `toString()`:

```
System.out.println(miCuenta);
// Salida legible: CuentaBancaria[IBAN='ES912100...', Titular='Sofía Morales', Saldo=1450.50
€, Bloqueada=false]
```

6.3 Comparación de Objetos: Operador `==` vs. Método `equals()`

Este es el punto donde más programadores noveles tropiezan:

```
CuentaBancaria c1 = new CuentaBancaria("Carlos", "ES001");
CuentaBancaria c2 = new CuentaBancaria("Carlos", "ES001");

System.out.println(c1 == c2);      // ¡Imprime FALSE!
System.out.println(c1.equals(c2)); // Imprime TRUE (si sobrescribimos equals correctamente)
```

¿Por qué ocurre esto?

- **El operador `==`:** Compara **identidad física de referencia** (si ambas variables apuntan a la misma dirección física en el Heap). Como `c1` y `c2` fueron creados con dos sentencias `new` distintas, residen en posiciones de memoria diferentes; por tanto `c1 == c2` es `false`.
- **El método `equals()`:** Compara **igualdad de contenido o semántica de negocio** (si dos objetos representan la misma entidad real, por ejemplo dos cuentas con el mismo IBAN o dos cartas con el mismo palo y valor).

El Contrato Riguroso de `equals()` en Java

Según la especificación oficial de la JVM, cualquier implementación de `equals` debe cumplir cinco propiedades matemáticas:

1. **Reflexiva:** `x.equals(x)` debe devolver siempre `true`.
2. **Simétrica:** `x.equals(y)` debe devolver `true` si y solo si `y.equals(x)` devuelve `true`.
3. **Transitiva:** Si `x.equals(y)` y `y.equals(z)` son `true`, entonces `x.equals(z)` debe ser `true`.
4. **Consistente:** Múltiples invocaciones sobre objetos no modificados deben arrojar siempre el mismo resultado.
5. **Tratamiento de `null`:** `x.equals(null)` debe devolver siempre `false` (sin lanzar `NullPointerException`).

Implementación Canónica Paso a Paso de `equals()` (Java Moderno):

```
import java.util.Objects;

public class Cliente {
    private final String dni;
    private String nombre;
    private String email;

    public Cliente(String dni, String nombre, String email) {
        this.dni = Objects.requireNonNull(dni);
        this.nombre = nombre;
        this.email = email;
    }

    @Override
    public boolean equals(Object obj) {
        // 1. Comprobación rápida de identidad física (¿apuntan a la misma dirección?)
        if (this == obj) {
            return true;
        }

        // 2. Comprobación de null, tipo compatible y cast seguro con Pattern Matching
        if (!(obj instanceof Cliente other)) {
            return false;
        }

        // 3. Comparación de los atributos clave (Objects.equals protege frente a posibles n
ulls)
        return Objects.equals(this.dni, other.dni);
    }

    @Override
    public int hashCode() {
        // La regla de oro: si dos objetos son iguales por equals, DEBEN devolver el mismo h
ashCode
        return Objects.hash(this.dni);
    }
}
```

(Nota: Para comparar atributos primitivos dentro de `equals`, se usa `==` para `int`, `long`, `boolean` o `char`, y `Double.compare(this.precio, other.precio) == 0` para `double`).

6.4 El Contrato Indisociable con `hashCode()` y la Metáfora de los Cajones

El método `hashCode()` devuelve un número entero (`int`) que resume el contenido del objeto. Las colecciones basadas en tablas hash (como `HashSet` o `HashMap`, que estudiaremos en la Unidad 6) utilizan este número para organizar los objetos en **"cajones" (*buckets*)** de búsqueda ultrarrápida:

1. Cuando insertas un objeto en un `HashSet`, Java calcula primero su `hashCode()` para decidir en qué cajón guardarlo.
2. Cuando preguntas si un objeto ya existe (`.contains(obj)`), Java **no recorre toda la colección**: calcula su `hashCode()`, va directamente a ese cajón concreto y **solo dentro de ese cajón** compara los objetos usando `equals()`.

La Regla de Oro de equals y hashCode

- **Si `a.equals(b)` es `true`, entonces `a.hashCode()` y `b.hashCode()` DEBEN devolver obligatoriamente el mismo número entero.**
- *(El inverso no es obligatorio: dos objetos distintos pueden coincidir por azar en el mismo `hashCode` —colisión de hash—, en cuyo caso `equals()` los desempatará dentro del cajón).*

Si sobrescribes `equals()` pero olvidas sobrescribir `hashCode()`, dos objetos con el mismo DNI tendrán distinto `hashCode` (calculado por defecto según su dirección de memoria), caerán en cajones diferentes y `HashSet` jamás detectará que están duplicados. **Usa siempre exactamente los mismos atributos en `Objects.equals(...)` y en `Objects.hash(...)`.**

6.5 Generación de Métodos Canónicos con IntelliJ IDEA

En el desarrollo profesional nadie pica a mano el código de `equals()` y `hashCode()`, pues es fácil cometer errores sutiles de verificación de nulos:

1. Dentro de tu clase en IntelliJ IDEA, presiona `Alt + Insert` (o clic derecho → *Generate...*).
2. Selecciona `equals() and hashCode()`.
3. Elige la plantilla *IntelliJ Default* o *java.util.Objects.equals and hashCode*.
4. Marca las casillas de los campos que identifican de forma única a tu entidad (por ejemplo, el `dni` o el `iban`).
5. Pulsa *Finish*. El IDE generará el código canónico optimizado y libre de errores.

---

7. Diseño Avanzado de Clases, Clases Especiales y Patrones

El dominio de la orientación a objetos no concluye con la creación de clases ordinarias. La arquitectura de software moderna exige patrones de diseño contrastados y mecanismos para garantizar la inmutabilidad y la claridad semántica.

7.1 Clases Inmutables: El Peligro de las Referencias Mutables y la Copia Defensiva

Un **objeto inmutable** es aquel cuyo estado no puede alterarse en absoluto tras su instanciación. Ejemplos eminentes en el JDK son `String`, `Integer`, `LocalDate` y `BigDecimal`.

Ventajas Fundamentales de la Inmutabilidad:

1. **Seguridad frente a hilos (*Thread-Safety*):** En aplicaciones multihilo no se necesitan bloques de sincronización (`synchronized`) ni cerrojos, ya que ningún hilo puede corromper un estado de solo lectura.
2. **Ausencia de efectos colaterales:** Puedes pasar un objeto inmutable a cualquier método con la garantía absoluta de que regresará intacto.
3. **Fiabilidad en colecciones:** Su `hashCode()` nunca cambia tras insertarse en un `HashSet` o `HashMap`.

El Error Clásico: La Implementación "Ingenua" (*Naive*) frente a la Copia Defensiva

Muchos programadores creen que basta con poner `private final` y quitar los setters para que una clase sea inmutable. Observa qué ocurre cuando la clase contiene un atributo cuyo tipo es un **objeto mutable** (como un array `int[] notas`):

```
// ❌ IMPLEMENTACIÓN INGENUA (VULNERABLE): ¡Fuga de referencias!
public class AlumnoInmutableRotura {
    private final String nombre;
    private final int[] notas; // El array en sí es un objeto MUTABLE en el Heap

    public AlumnoInmutableRotura(String nombre, int[] notas) {
        this.nombre = nombre;
        this.notas = notas; // ¡PELIGRO 1! Guarda la misma referencia del array externo
    }

    public int[] getNotas() {
        return this.notas;  // ¡PELIGRO 2! Devuelve la referencia directa al array interno
    }
}
```

Si alguien hace `int[] misNotas = {8, 9}; AlumnoInmutableRotura a = new AlumnoInmutableRotura("Ana", misNotas);`, y acto seguido escribe `misNotas[0] = 0;` o `a.getNotas()[1] = 0;`, ¡habrá modificado las notas internas del supuesto objeto inmutable!

Las 4 Reglas de Oro para Blindar una Clase Inmutable:

1. Declarar la clase como **`final`** (`public final class ...`) para impedir que una subclase herede de ella y rompa la inmutabilidad.
2. Declarar todos los atributos como **`private final`**.
3. **No definir ningún método setter** ni métodos que alteren el estado (si se requiere una operación de cambio, debe devolver una **nueva instancia**).
4. **Realizar copias defensivas (`.clone()` o `Arrays.copyOf()`)** de cualquier objeto mutable tanto al **entrar** por el constructor como al **salir** por un getter:

```
import java.util.Arrays;

// ✅ IMPLEMENTACIÓN BLINDADA Y VERDADERAMENTE INMUTABLE
public final class AlumnoInmutable {
    private final int id;
    private final String nombre; // String ya es inmutable por naturaleza
    private final int[] notas;   // Array mutable: exige copia defensiva en entrada y salida

    public AlumnoInmutable(int id, String nombre, int[] notas) {
        this.id = id;
        this.nombre = (nombre != null) ? nombre : "Sin nombre";
        // COPIA DEFENSIVA DE ENTRADA (con .clone() o Arrays.copyOf):
        this.notas = (notas != null) ? notas.clone() : new int[0];
    }

    public int getId() { return this.id; }
    public String getNombre() { return this.nombre; }

    public int[] getNotas() {
        // COPIA DEFENSIVA DE SALIDA: devuelve un clon independiente
        return this.notas.clone();
    }

    @Override
    public String toString() {
        return "AlumnoInmutable[id=" + id + ", nombre='" + nombre + "', notas=" + Arrays.toS
tring(notas) + "]";
    }
}
```

7.2 Los `record` en Java Moderno (Java 16+, 21 LTS y 25 LTS)

En aplicaciones corporativas, una gran parte de las clases son meros **transportadores de datos inmutables** (*Data Carriers* o DTOs - *Data Transfer Objects*). Escribir manualmente constructores, getters, `equals()`, `hashCode()` y `toString()` genera decenas de líneas de código repetitivo (*boilerplate*).

Para solucionar esto, Java moderno introdujo los **`record`**: una forma concisa y declarativa de definir clases de datos inmutables:

```
// ¡Una única línea sustituye a 50 líneas de código tradicional!
public record CoordenadaGPS(double latitud, double longitud, String descripcion) {

    // Constructor compacto opcional para comprobación de invariantes y precondiciones
    public CoordenadaGPS {
        if (latitud < -90.0 || latitud > 90.0) {
            throw new IllegalArgumentException("Latitud fuera de rango [-90, 90]: " + latitu
d);
        }
        if (longitud < -180.0 || longitud > 180.0) {
            throw new IllegalArgumentException("Longitud fuera de rango [-180, 180]: " + lon
gitud);
        }
        if (descripcion == null) {
            descripcion = "Sin descripción";
        }
    }
}
```

¿Qué Genera Automáticamente la JVM para un `record`?

1. Una clase `final` inmutable que hereda de `java.lang.Record`.
2. Todos los campos declarados como componentes `private final`.
3. Un constructor canónico con todos los componentes.
4. Métodos accesores con el **mismo nombre que el componente, sin prefijo** `get` (por ejemplo: `punto.latitud()`, `punto.longitud()`).
5. Implementaciones canónicas perfectas de `equals()`, `hashCode()` y `toString()`.

7.3 El Patrón de Diseño Singleton (Inicialización Temprana vs. Perezosa)

El **Singleton** es un patrón de diseño creacional cuyo objetivo es garantizar que una clase tenga **una única instancia viva en toda la aplicación**, proporcionando un punto de acceso global a ella (por ejemplo, un gestor de configuración, un registro de logs o una caché central).

Se basa siempre en tres pilares:

1. **Constructor** `private` para impedir que nadie haga `new` desde el exterior.
2. **Atributo** `private static` del mismo tipo de la clase que guarda la referencia al objeto único.
3. **Método** `public static getInstancia()` que devuelve siempre ese único objeto.

Existen tres formas clásicas de implementarlo según cuándo y cómo se crea la instancia:

Variante A: Inicialización Perezosa (*Lazy Initialization*)

El objeto único no se crea al arrancar el programa, sino **solo en el instante en que alguien invoca** `getInstancia()` **por primera vez**:

```
public class Configuracion {
    // 1. Referencia estática inicialmente nula
    private static Configuracion instancia;
    private String idioma;

    // 2. Constructor privado: prohibido hacer 'new Configuracion()' desde fuera
    private Configuracion() {
        this.idioma = "es";
    }

    // 3. Método de acceso global: crea el objeto solo en la primera llamada
    public static Configuracion getInstancia() {
        if (instancia == null) {
            instancia = new Configuracion();
        }
        return instancia;
    }

    public String getIdioma() { return this.idioma; }
    public void setIdioma(String idioma) { this.idioma = idioma; }
}
```

Comprobación de identidad única:

```
Configuracion c1 = Configuracion.getInstancia();
Configuracion c2 = Configuracion.getInstancia();

c1.setIdioma("en");
System.out.println(c2.getIdioma()); // Imprime: "en"
System.out.println(c1 == c2);       // Imprime: true (son exactamente el mismo objeto en el
Heap)
```

Variante B: Inicialización Temprana (*Eager Initialization*) y Seguridad Multihilo

En aplicaciones con múltiples hilos (*threads*), la inicialización perezosa básica puede fallar si dos hilos entran a la vez en `if (instancia == null)`. Para garantizar seguridad total sin bloqueos:

- Podemos añadir `synchronized` a la cabecera (`public static synchronized Configuracion getInstancia()`), o bien:
- Crear la instancia directamente en una constante `private static final` (**Eager Initialization**) o mediante una clase interna estática (*Holder Pattern*), donde la propia JVM garantiza por diseño la creación atómica de una sola instancia:

```
public class ConfiguracionSistema {
    // Instancia única creada de forma segura al cargar la clase (Eager Initialization)
    private static final ConfiguracionSistema INSTANCIA = new ConfiguracionSistema();

    private String urlServidor = "https://api.empresa.es";

    private ConfiguracionSistema() {}

    public static ConfiguracionSistema getInstancia() {
        return INSTANCIA;
    }

    public String getUrlServidor() { return this.urlServidor; }
}
```

7.4 Enumeraciones (`enum`): De las Constantes Simbólicas a Objetos de Pleno Derecho

Imagina una empresa de alquiler de vehículos donde alquilar un coche cuesta `100.0 €` y una moto `50.0 €`.

- Si codificamos los tipos con números enteros (`1` para coche, `2` para moto) o constantes sueltas (`public static final int COCHE = 1;`), cualquier método `getPrecioAlquiler(int tipo)` nos obliga a validar manualmente que nadie pase un `99` o un `-4`.
- En Java, una **enumeración (`enum`) es una clase completa** donde cada constante declarada es, internamente, una instancia `public static final` de esa clase:

```
// Definir enum Vehiculo { COCHE, MOTO } equivale conceptualmente a:
// class Vehiculo {
//     public static final Vehiculo COCHE = new Vehiculo();
//     public static final Vehiculo MOTO  = new Vehiculo();
// }
```

Al ser una clase real, un `enum` puede tener **atributos `private final`, constructores privados y métodos getters o de negocio**:

```
public enum Vehiculo {
    // Al declarar cada constante se invoca implícitamente al constructor privado
    COCHE(100.0, "Turismo de 5 plazas"),
    MOTO(50.0, "Motocicleta urbana"),
    FURGONETA(140.0, "Vehículo industrial de carga");

    private final double precioDia;
    private final String descripcion;

    // El constructor de un enum es SIEMPRE privado
    Vehiculo(double precioDia, String descripcion) {
        this.precioDia = precioDia;
        this.descripcion = descripcion;
    }

    public double getPrecioDia() { return this.precioDia; }
    public String getDescripcion() { return this.descripcion; }
}
```

Métodos Nativos de todo `enum`:

- `Vehiculo.values()`: Devuelve un array `Vehiculo[]` con todas las constantes definidas.
- `Vehiculo.valueOf("COCHE")`: Devuelve la constante `Vehiculo.COCHE` a partir de su texto (o lanza `IllegalArgumentException` si no existe).
- `v.name()`: Devuelve el nombre exacto de la constante (`"COCHE"`).
- `v.ordinal()`: Devuelve el índice entero de posición (`0` para `COCHE`, `1` para `MOTO`, `2` para `FURGONETA`).

7.5 Anotaciones en Java (`@interface`): Metadatos para el Compilador y Tiempo de Ejecución

Las **anotaciones** son metadatos (como un *post-it* informático prefijado con `@`) que se incrustan sobre clases, métodos, atributos o parámetros. No alteran por sí mismas el flujo de instrucciones, pero proporcionan información vital al compilador, a herramientas de generación de código o al propio programa en tiempo de ejecución (vía *Reflection*).

Las 9 Anotaciones Estándar del Lenguaje Java:

1. **Anotaciones para el compilador (`java.lang`):**
   - `@Override`: Verifica en compilación que el método sobrescribe realmente a uno de su superclase o interfaz.
   - `@Deprecated`: Marca un método o clase como obsoleto para desaconsejar su uso (el IDE lo muestra tachado).
   - `@SuppressWarnings("...")`: Indica al compilador que silencie advertencias específicas conocidas.
   - `@FunctionalInterface`: Garantiza que una interfaz tenga exactamente un único método abstracto (para expresiones lambda).
   - `@SafeVarargs`: Suprime advertencias de seguridad en métodos con argumentos variables genéricos (`T...`).
2. **Meta-anotaciones (`java.lang.annotation` — sirven para configurar nuevas anotaciones):**
   - `@Target`: Define dónde se puede colocar la anotación (`ElementType.TYPE`, `FIELD`, `METHOD`, `PARAMETER`...).
   - `@Retention`: Define hasta qué fase sobrevive la anotación (`SOURCE`, `CLASS` o `RUNTIME` para leerla en ejecución).
   - `@Documented` e `@Inherited`: Incluyen la anotación en Javadoc y permiten que las subclases la hereden.

Creación y Procesamiento de una Anotación Propia con la API Reflection:

Cuando trabajes en el futuro con frameworks profesionales como *Spring Boot*, *Hibernate/JPA* o *JUnit*, verás anotaciones como `@Entity`, `@Column` o `@Test`. Así es como funcionan por dentro:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;

// 1. Definimos nuestra propia anotación @Imprimible
@Target({ElementType.FIELD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME) // Disponible en tiempo de ejecución para Reflection
@interface Imprimible {
    boolean mayusculas() default false;
}

// 2. Aplicamos la anotación a los atributos de una clase
class LibroAnotado {
    @Imprimible(mayusculas = false)
    String titulo;

    @Imprimible(mayusculas = true)
    String autor;

    public LibroAnotado(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
    }
}

// 3. Procesamos los metadatos dinámicamente usando Reflection
public class ProcesadorAnotaciones {
    public static void imprimirCamposAnotados(Object objeto) throws IllegalAccessException {
        for (Field campo : objeto.getClass().getDeclaredFields()) {
            Imprimible anotacion = campo.getAnnotation(Imprimible.class);
            if (anotacion != null) {
                String valor = String.valueOf(campo.get(objeto));
                System.out.println(anotacion.mayusculas() ? valor.toUpperCase() : valor);
            }
        }
    }
}
```

---

8. Relaciones Básicas entre Objetos: Agregación Simple y Arrays de Objetos

Hasta ahora hemos creado clases individuales que contienen atributos de tipos primitivos (`int`, `double`, `boolean`) o tipos inmutables como `String`. No obstante, el auténtico poder del paradigma orientado a objetos surge cuando **los objetos colaboran, se combinan y se contienen unos a otros** para modelar fielmente la complejidad del mundo real.

8.1 El Principio «Tiene-un» (*Has-a*): Agregación y Composición Simple

En el diseño de software orientado a objetos, una clase puede tener como atributo una referencia a otra clase. Esta relación se conoce genéricamente como **asociación estructural** y responde a la pregunta semántica: **«¿Un objeto A *tiene un* objeto B?»**:

- Un `Vehiculo` *tiene un* `Motor`.

- Un `Alumno` *tiene una* `Direccion`.
- Una `Factura` *tiene un* `Cliente`.

```
public class Motor {
    private String tipoCombustible; // "Gasolina", "Diésel", "Eléctrico"
    private int potenciaCv;
    private boolean encendido;

    public Motor(String tipoCombustible, int potenciaCv) {
        this.tipoCombustible = tipoCombustible;
        this.potenciaCv = potenciaCv;
        this.encendido = false;
    }

    public void encender() { this.encendido = true; }
    public void apagar() { this.encendido = false; }
    public boolean isEncendido() { return encendido; }
    public int getPotenciaCv() { return potenciaCv; }
}

public class Vehiculo {
    private String matricula;
    private String marca;
    private Motor motor; // ATRIBUTO DE TIPO OBJETO: Agregación simple ("Tiene-un")

    public Vehiculo(String matricula, String marca, Motor motor) {
        if (matricula == null || motor == null) {
            throw new IllegalArgumentException("Matrícula y motor son obligatorios");
        }
        this.matricula = matricula;
        this.marca = marca;
        this.motor = motor; // Conecta el motor al vehículo
    }

    // El vehículo delega en su motor interno las operaciones mecánicas
    public void arrancar() {
        System.out.println("Poniendo en marcha el vehículo " + marca + "...");
        motor.encender();
    }

    public void detener() {
        System.out.println("Deteniendo el vehículo " + marca + "...");
        motor.apagar();
    }
}
```

Delegación de Responsabilidades

Fíjate en cómo `Vehiculo` no manipula directamente cables ni bujías: simplemente invoca `motor.encender()`. Esta técnica se denomina **delegación** y es la base de un código modular, donde cada clase es experta únicamente en su propia parcela de responsabilidad.

8.2 Gestión de Múltiples Objetos: Arrays de Objetos como Atributo de Clase

Antes de estudiar las estructuras dinámicas de la biblioteca de colecciones (`ArrayList`, `HashSet`) en la Unidad 6, en el desarrollo profesional con Java es vital aprender a modelar **clases contenedoras de dominio** que gestionan internamente una colección de objetos mediante un array convencional:

Imagina que necesitamos modelar un `Catalogo` que almacene hasta un máximo de `N` libros:

```
public class CatalogoLibros {
    private Libro[] libros;     // Array que almacena las referencias a los objetos
    private int totalLibros;    // Contador de cuántos libros reales hay almacenados

    public CatalogoLibros(int capacidadMaxima) {
        if (capacidadMaxima <= 0) {
            throw new IllegalArgumentException("La capacidad debe ser mayor que cero");
        }
        this.libros = new Libro[capacidadMaxima]; // Reserva el vector en el Heap
        this.totalLibros = 0;
    }

    // 1. ALTA SEGURA DE UN NUEVO OBJETO
    public boolean agregarLibro(Libro nuevoLibro) {
        if (nuevoLibro == null) {
            throw new IllegalArgumentException("No se puede agregar un libro nulo");
        }
        if (totalLibros >= libros.length) {
            return false; // Catálogo lleno
        }
        libros[totalLibros] = nuevoLibro;
        totalLibros++;
        return true;
    }

    // 2. BÚSQUEDA POR CRITERIO (ej. por título)
    public Libro buscarPorTitulo(String tituloBuscado) {
        if (tituloBuscado == null) return null;
        for (int i = 0; i < totalLibros; i++) {
            if (libros[i].getTitulo().equalsIgnoreCase(tituloBuscado)) {
                return libros[i]; // Retorna la referencia encontrada
            }
        }
        return null; // No encontrado
    }

    // 3. ELIMINACIÓN Y COMPACTACIÓN DEL ARRAY
    public boolean eliminarPorTitulo(String tituloBuscado) {
        for (int i = 0; i < totalLibros; i++) {
            if (libros[i].getTitulo().equalsIgnoreCase(tituloBuscado)) {
                // Desplaza todos los libros siguientes un hueco a la izquierda para no deja
r huecos nulos
                for (int j = i; j < totalLibros - 1; j++) {
                    libros[j] = libros[j + 1];
                }
                libros[totalLibros - 1] = null; // Limpia la última referencia para el Garba
ge Collector
                totalLibros--;
                return true;
            }
        }
        return false;
    }

    public int getTotalLibros() { return totalLibros; }
    public int getCapacidadMaxima() { return libros.length; }
}
```

8.3 Métodos Factoría Estáticos (*Static Factory Methods*)

Los **métodos factoría estáticos** son métodos `public static` que instancian y devuelven un objeto de su propia clase. Representan una alternativa de diseño muy superior a los constructores sobrecargados en diversas situaciones:

1. **Tienen nombres descriptivos:** Mientras que todos los constructores deben llamarse forzosamente igual que la clase, un método factoría explica claramente *cómo* se crea el objeto (ej. `Punto2D.desdeCartesianas(x, y)` frente a `Punto2D.desdePolares(radio, angulo)`).
2. **Evitan colisiones de firma:** Si dos formas de instanciar un objeto requieren los mismos tipos de parámetros (por ejemplo dos `double`), no es posible crear dos constructores distintos. La factoría estática resuelve esto con elegancia.
3. **No están obligados a crear una nueva instancia cada vez:** Pueden devolver instancias inmutables compartidas o precalculadas en caché (como hace `Integer.valueOf(int)` o `Boolean.valueOf(boolean)`).

```
public final class ColorRGB {
    private final int rojo;
    private final int verde;
    private final int azul;

    // Constantes predefinidas compartidas
    public static final ColorRGB ROJO = new ColorRGB(255, 0, 0);
    public static final ColorRGB VERDE = new ColorRGB(0, 255, 0);
    public static final ColorRGB AZUL = new ColorRGB(0, 0, 255);
    public static final ColorRGB BLANCO = new ColorRGB(255, 255, 255);
    public static final ColorRGB NEGRO = new ColorRGB(0, 0, 0);

    // Constructor privado: NADIE desde fuera puede hacer 'new ColorRGB(...)'
    private ColorRGB(int rojo, int verde, int azul) {
        this.rojo = rojo;
        this.verde = verde;
        this.azul = azul;
    }

    // FACTORÍA ESTÁTICA 1: Desde valores numéricos [0-255]
    public static ColorRGB fromRGB(int r, int g, int b) {
        if (r < 0 || r > 255 || g < 0 || g > 255 || b < 0 || b > 255) {
            throw new IllegalArgumentException("Componentes RGB deben estar entre 0 y 255");
        }
        return new ColorRGB(r, g, b);
    }

    // FACTORÍA ESTÁTICA 2: Desde una cadena hexadecimal ("#FF5733")
    public static ColorRGB fromHex(String hex) {
        if (hex == null || !hex.matches("^#?[0-9a-fA-F]{6}$")) {
            throw new IllegalArgumentException("Formato hexadecimal inválido: " + hex);
        }
        String limpio = hex.startsWith("#") ? hex.substring(1) : hex;
        int r = Integer.parseInt(limpio.substring(0, 2), 16);
        int g = Integer.parseInt(limpio.substring(2, 4), 16);
        int b = Integer.parseInt(limpio.substring(4, 6), 16);
        return new ColorRGB(r, g, b);
    }

    public int getRojo() { return rojo; }
    public int getVerde() { return verde; }
    public int getAzul() { return azul; }
}
```

8.4 Aplicación de Expresiones Regulares en la Validación de Objetos

En la Unidad 3 estudiamos en profundidad la sintaxis y el motor de **Expresiones Regulares (*Regex*)** de Java. En la Programación Orientada a Objetos, las expresiones regulares se convierten en la principal línea de defensa para **garantizar las invariantes de los objetos** en sus constructores, setters y `records`:

```
public record Usuario(String username, String email, String nif) {
    // Constructor compacto con validaciones rigurosas mediante Regex aprendidas en UD3
    public Usuario {
        if (username == null || !username.matches("^[a-zA-Z0-9_]{4,15}$")) {
            throw new IllegalArgumentException("Username inválido: debe ser alfanumérico ent
re 4 y 15 caracteres");
        }
        if (email == null || !email.matches("^[\\w.-]+@[\\w.-]+\\.[a-zA-Z]{2,}$")) {
            throw new IllegalArgumentException("Formato de correo electrónico no válido: " +
email);
        }
        if (nif == null || !nif.matches("^\\d{8}[A-HJ-NP-TV-Z]$")) {
            throw new IllegalArgumentException("Formato de DNI español incorrecto: " + nif);
        }
    }
}
```

Gracias a esta integración, ningún objeto `Usuario` podrá crearse en el Heap con un correo corrupto o un identificador ilegítimo, blindando la integridad de todo el sistema.

---

9. Organización Modular: Paquetes e Importaciones

A medida que una aplicación crece y cuenta con decenas o cientos de clases, es indispensable organizarlas en carpetas lógicas para evitar el caos y prevenir colisiones de nombres.

9.1 Concepto de Paquete (`package`)

Un **paquete** en Java cumple una doble función:

1. **Espacio de nombres (*Namespace*):** Permite que existan dos clases con el mismo nombre siempre que pertenezcan a paquetes diferentes (por ejemplo, `java.util.Date` frente a `java.sql.Date`).
2. **Estructura física en disco:** La sentencia de paquete debe coincidir **exactamente con la jerarquía de carpetas en el sistema de archivos**.

Convención Oficial de Nombres de Paquetes:

Se utiliza el **nombre de dominio inverso de la organización o centro educativo** en minúsculas puras, seguido del nombre del proyecto y el subdominio funcional:

```
package es.iesperenxisa.dam.gestionbancaria.modelo;
```

Físicamente en el disco duro, este fichero debe residir en la ruta:
`src/main/java/es/iesperenxisa/dam/gestionbancaria/modelo/CuentaBancaria.java`

9.2 Sentencias `import` e Importaciones Estáticas

Para utilizar una clase situada en otro paquete, debemos indicárselo al compilador mediante la instrucción **`import`**:

```
import java.util.Scanner;        // Importación explícita (Recomendada)
import java.time.LocalDate;

// Importación con comodín (importa todas las clases públicas del paquete, pero no sus subpa
quetes)
import java.util.*;

// Importación estática: permite invocar métodos y constantes estáticas directamente sin ant
eponer la clase
import static java.lang.Math.PI;
import static java.lang.Math.sqrt;

public class CalculoGeometria {
    public double calcularAreaCirculo(double r) {
        return PI * r * r; // Se usa PI directamente en vez de Math.PI
    }
}
```

> **El Paquete Implícito java.lang**
> Todas las clases del paquete fundamental **`java.lang`** (como `String`, `Math`, `System`, `Integer`, `Double`, `Object` o `Exception`) son importadas de forma completamente automática e invisible por el compilador en todos los ficheros de Java. Nunca necesitas escribir `import java.lang.String;`.

---

10. ⭐ Be the Code: Desafíos de Traza y Memoria

Un programador que domina la POO debe visualizar instantáneamente qué ocurre en la memoria Stack y Heap ante cada sentencia. Resuelve mentalmente los siguientes tres retos.

Desafío 1: Comparaciones de Identidad vs. Contenido y Paso por Referencia

Analiza minuciosamente el siguiente código y deduce qué imprime exactamente en la consola:

```
public class RetoPOO1 {
    static class Mensaje {
        String texto;
        Mensaje(String t) { this.texto = t; }
    }

    public static void alterar(Mensaje m1, Mensaje m2) {
        m1.texto = "Modificado";
        m2 = new Mensaje("Nuevo");
        m2.texto = "Cambiado";
    }

    public static void main(String[] args) {
        Mensaje a = new Mensaje("Original A");
        Mensaje b = new Mensaje("Original B");
        Mensaje c = a;

        alterar(a, b);

        System.out.println("1: " + a.texto);
        System.out.println("2: " + b.texto);
        System.out.println("3: " + (a == c));
        System.out.println("4: " + a.equals(c));
    }
}
```

▶ 🔍 Ver Resolución y Traza de Memoria

Desafío 2: Encadenamiento de Constructores y Atributos Estáticos

¿Cuál es la salida producida en la consola tras ejecutar este programa?

```
public class RetoPOO2 {
    static class ContadorInstancias {
        static int total = 0;
        int id;
        String nombre;

        ContadorInstancias(String n) {
            this(n, ++total);
            System.out.print("C1(" + this.nombre + ") ");
        }

        ContadorInstancias(String n, int i) {
            this.nombre = n;
            this.id = i;
            System.out.print("C2(" + this.id + ") ");
        }
    }

    public static void main(String[] args) {
        ContadorInstancias obj1 = new ContadorInstancias("Alfa");
        ContadorInstancias obj2 = new ContadorInstancias("Beta", 99);
        System.out.println("-> Total: " + ContadorInstancias.total);
    }
}
```

▶ 🔍 Ver Resolución y Traza

Desafío 3: La Trampa de la Mutabilidad Encubierta

Observa la siguiente clase y determina qué imprimirá la última línea:

```
public class RetoPOO3 {
    static class BovedaSegura {
        private final int[] claves;

        public BovedaSegura(int[] clavesEntrada) {
            this.claves = clavesEntrada; // ¡Alerta de diseño!
        }

        public int[] getClaves() {
            return this.claves; // ¡Alerta de diseño!
        }
    }

    public static void main(String[] args) {
        int[] misClaves = {1234, 5678};
        BovedaSegura boveda = new BovedaSegura(misClaves);

        misClaves[0] = 9999;
        boveda.getClaves()[1] = 0000;

        System.out.println(java.util.Arrays.toString(boveda.getClaves()));
    }
}
```

▶ 🔍 Ver Resolución y Explicación de Fuga de Referencia

---

11. ¡No Hay Preguntas Tontas!

---

1. ¿Por qué el constructor tiene que llamarse exactamente igual que la clase y no tiene tipo de retorno?

Porque el constructor no es una función ordinaria que calcula y devuelve un valor, sino una rutina especializada de inicialización de memoria gestionada directamente por el operador `new`. La JVM necesita una convención inequívoca para distinguir la creación de un objeto de una llamada a método. Si tuviera tipo de retorno (incluso `void`), el compilador creería que es un método cualquiera que casualmente comparte el nombre con la clase.

---

2. ¿Por qué no dejamos todos los atributos públicos y nos ahorramos escribir getters y setters?

Si dejas los atributos `public`, cualquier parte del programa puede asignar valores destructivos o ilegales en cualquier momento (como `cuenta.saldo = -999999;` o `alumno.nota = 450.0;`) sin que la clase pueda impedirlo ni enterarse. Al hacerlos `private` y usar `getters` y `setters`, el objeto mantiene el **control absoluto sobre sus invariantes**: puede validar datos, lanzar excepciones defensivas, calcular campos al vuelo o mantener auditorías de quién modificó la información.

---

3. Si pongo `final` en una variable objeto (`final Cuenta c = new Cuenta()`), ¿significa que su saldo no puede cambiar?

No. Este es uno de los equívocos más comunes. La palabra `final` en una variable de referencia significa que **la referencia (la dirección de memoria)** no puede cambiar jamás: nunca podrás reasignar `c = new Cuenta()`. Sin embargo, el objeto vivo en el Heap al que apunta `c` sigue siendo completamente mutable: puedes llamar a `c.ingresar(500)` y alterar su saldo libremente. Para que el estado interno sea inmutable, deben ser los atributos internos de la clase los que se declaren como `final`.

---

4. ¿Por qué `c1.equals(c2)` devuelve `false` por defecto si no lo sobrescribo?

Porque la implementación original de `equals()` en la clase raíz `java.lang.Object` se define literalmente como `return (this == obj);`. Es decir, si no sobrescribes el método en tu clase, `equals()` se comporta con la misma rigidez que el operador `==`, comparando únicamente si ambos objetos ocupan la misma dirección física en el Heap.

---

5. ¿Cuál es la diferencia conceptual entre una variable de instancia y una variable `static`?

- **Variable de instancia:** Pertenece a **un objeto concreto**. Si creas 100 objetos `CuentaBancaria`, habrá 100 variables `saldo` distintas en el Heap, cada una con el dinero de su titular.
- **Variable `static`:** Pertenece a **la clase global**. Existe una única celda de memoria en toda la JVM compartida por los 100 objetos (por ejemplo, el contador de cuántas cuentas existen en el banco).

---

6. ¿Por qué dentro de un método `static` no puedo utilizar la palabra reservada `this`?

Porque un método estático se ejecuta a nivel de clase (`NombreClase.metodo()`), con independencia de que existan o no objetos instanciados en el Heap. La palabra `this` representa la dirección del objeto que está ejecutando la llamada; en un contexto estático, no hay ningún objeto en ejecución, por lo que `this` carece por completo de significado físico.

---

7. ¿Qué es exactamente un `record` de Java y cuándo debo usarlo en lugar de una clase normal?

Un `record` es una clase especializada inmutable pensada para actuar como transportador transparente de datos. Debes usar un `record` cuando necesites agrupar un conjunto fijo de datos de solo lectura (como una coordenada, una fila leída de base de datos, un par clave-valor o un resultado de cálculo). Debes seguir usando una `class` ordinaria cuando tu entidad requiera estado mutable, herencia jerárquica con `extends` o encapsulamiento con lógica de negocio compleja.

---

8. ¿Por qué en una enumeración (`enum`) el constructor es obligatoriamente privado?

Porque el propósito de un `enum` es definir un conjunto exhaustivo y cerrado de instancias fijas conocidas en tiempo de compilación (`LUNES`, `MARTES`, etc.). Si el constructor fuera público, cualquier parte del programa podría ejecutar `new DiaSemana()` en tiempo de ejecución, creando días de la semana inexistentes y destruyendo la garantía de conjunto finito y seguro del tipo.

---

12. Resumen de la Unidad y Conexión con la Unidad 5

En esta Unidad 4 hemos dado el paso decisivo hacia la ingeniería del software orientada a objetos:

```
+-------------------------------------------------------------------------+
|                  NÚCLEO DE LA PROGRAMACIÓN ORIENTADA A OBJETOS          |
+-------------------+--------------------+------------------+-------------+
|  CLASES Y OBJETOS |  ENCAPSULAMIENTO   | MIEMBROS STATIC  | CLASE OBJECT|
| Molde en código   | Atributos private, | Variables de     | toString,   |
| vs instancias en  | getters/setters con| clase, constantes| equals y    |
| Heap con estado,  | validación, final  | globales y       | hashCode    |
| conducta y this.  | y records.         | utilidades.      | canónicos.  |
+-------------------+--------------------+------------------+-------------+
```

El Salto Hacia la Unidad 5: POO Avanzada

Ahora que sabemos crear clases robustas y encapsuladas, en la **Unidad 5: Programación orientada a objetos avanzada** exploraremos los otros dos grandes pilares de la POO:

- **Herencia (`extends`):** Cómo estructurar jerarquías de clases compartiendo atributos y métodos comunes mediante superclases y subclases.
- **Polimorfismo:** Cómo tratar a distintos objetos especializados bajo una misma categoría general.
- **Clases Abstractas e Interfaces (`interface` / `implements`):** El diseño formal de contratos y desacoplamiento en Java moderno.

---

13. Relación Curricular Oficial

Esta unidad cubre exhaustivamente los contenidos y criterios de evaluación prescritos en el currículo oficial del ciclo formativo **CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)** para el módulo **0485: Programación**:

- **RA1. Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.**
  - *CE a, d, f:* Identificación de bloques de clases, uso de operadores y comentarios de documentación Javadoc.
- **RA2. Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.**
  - *CE a, b, c, d, e, f, g, h, i, j:* Fundamentos de POO, instanciación en el Heap, invocación de métodos y propiedades, uso de miembros `static`, constructores y depuración con IntelliJ IDEA.
- **RA3. Escribe y optimiza código estructurado aplicando los principios de la programación modular.**
  - *CE a, b, c, d:* Modularización en clases, paso de parámetros de objetos por valor en el Heap, sobrecarga de constructores y métodos, y recursividad.
- **RA4. Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.**
  - *CE a, b, c, d, f:* Sintaxis y estructura de clases, definición de atributos y métodos, creación de constructores canónicos y sobrecargados, control de visibilidad (`private` , `public` , `protected` , package-private) e instanciación de objetos en aplicaciones integradas.

---

14. Boletines de Ejercicios y Retos Prácticos

Para afianzar y llevar a la práctica las habilidades de diseño y codificación orientadas a objetos, dispones de una batería completa y unificada de 5 boletines:

Recurso	Nivel / Enfoque	Descripción
📝 **Boletín 04: Nivel Inicial**	Básico / Consolidación	16 ejercicios progresivos: creación de clases (`Libro` , `Rectangulo` , `Punto2D` , `Empleado`), atributos `private` vs `public` , referencias compartidas, `this(...)` , `static` y sobrecarga.
✅ **Boletín 04: Inicial Resuelto**	Soluciones Comentadas	Soluciones íntegras y razonadas con código Java 25 LTS, trazas de memoria y validaciones.
🚀 **Boletín 04: Nivel Intermedio**	Intermedio / Aplicado	16 ejercicios de arquitectura POO: `toString` / `equals` / `hashCode` , clases inmutables (`Coordenada` , `AlumnoInmutable`), `record` , Singleton (`ContadorGlobal` , `GestorConfiguracion`), `Racional` , arrays de objetos (`Circulo`), `StringBuilder` y Regex.
✅ **Boletín 04: Intermedio Resuelto**	Soluciones Comentadas	Soluciones completas con demostración de contratos de igualdad, copias defensivas y patrones de diseño.
🏆 **Boletín 04: Retos Orientados a Objetos (Extras)**	Avanzado / Arquitectura	8 grandes retos: `Fraccion` con MCD recursivo, `Mazo` de 40 cartas con Fisher-Yates, `ConjuntoEnteros` con varargs, clínica dietista con ficheros, máquina expendedora y referencias a CodeWars y Acepta el Reto.